

第 9 章 数组运算

前面处理一批数字都是用列表加循环。数据一多，循环写起来啰嗦、跑起来也慢。这一章换成 NumPy 的数组，同样的事情能少写很多代码，也快得多。它还是后面所有数据分析和机器学习工具的地基。

装它只要一行：

```
pip install numpy
```

9.1 先算一个班的成绩

本节任务：一个班五名学生三门课的成绩，算出每人总分、每科平均分，并挑出总分最高的人。

```
1 import numpy as np
2
3 names = np.array(["小明", "小红", "小刚", "小美", "小强"])
4 # 每行一个人，每列一门课，依次是语文 数学 英语
5 scores = np.array([[86, 92, 78],
6                    [95, 88, 91],
7                    [55, 61, 48],
8                    [78, 85, 90],
9                    [90, 79, 83]])
10
11 print("形状：", scores.shape)
12 print("每人总分：", scores.sum(axis=1))
13 print("每科平均：", scores.mean(axis=0))
14 print("每人最高分：", scores.max(axis=1))
15
16 best = scores.sum(axis=1).argmax()
17 print("总分最高：", names[best], scores.sum(axis=1)[best])
18 print("及格的人次：", (scores >= 60).sum())
```

运行结果：

```
形状： (5, 3)
每人总分： [256 274 164 253 252]
每科平均： [80.8 81. 78. ]
每人最高分： [92 95 61 90 90]
总分最高： 小红 274
```

及格的人次： 13

注意一个循环都没写。求和、平均、最大值、比较、计数全是一句话。这就是数组的 ★核心价值，把对一批数的操作写成一次运算，而不是一个个遍历。下面把这些用法讲清楚。

9.2 数组和列表有什么不同

本节任务：搞清为什么要用数组而不是列表。

用 `np.array` 可以把列表变成数组：

```
1 import numpy as np
2 a = np.array([1, 2, 3, 4])
3 print(a) # [1 2 3 4]
4 print(type(a)) # <class 'numpy.ndarray'>
```

看着像列表，但有三点关键区别。

第一，数组能整体运算。列表乘 2 是把自己重复一遍，数组乘 2 是每个元素都乘 2：★核心

```
1 print([1, 2, 3] * 2) # [1, 2, 3, 1, 2, 3]
2 print(np.array([1, 2, 3]) * 2) # [2 4 6]
3 print(np.array([1, 2, 3]) + 10) # [11 12 13]
4 print(np.array([1, 2, 3]) ** 2) # [1 4 9]
```

两个等长数组之间也能直接运算，按位置一一对应：

```
1 a = np.array([1, 2, 3])
2 b = np.array([10, 20, 30])
3 print(a + b) # [11 22 33]
4 print(a * b) # [10 40 90]
```

第二，数组里的元素类型必须一致，全是整数或全是小数。列表可以混装，数组不行。这个限制换来了速度，同类型的数据在内存里排得整整齐齐，运算能一次处理一批。

第三，数组有形状。列表只能是一串，数组可以是二维的表、三维的块。开头那个成绩表就是五行三列的二维数组。

9.3 形状和维度

本节任务：会看形状、改形状，理解按行还是按列计算。

用 `shape` 看形状，`reshape` 改形状：

```
1 a = np.arange(12) # 0 到 11
```

```

2 print(a.shape) # (12,) 一维, 12 个
3 b = a.reshape(3, 4) # 变成 3 行 4 列
4 print(b.shape) # (3, 4)
5 print(b)

```

输出:

```

[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]

```

reshape 时元素总数必须对得上, 3 乘 4 等于 12 才行。想让它自己算某一维, 那一维写 -1:

```

1 print(a.reshape(2, -1).shape) # (2, 6) 自动算出 6

```

二维数组求和求平均时, 要指定沿哪个方向算, 这靠 axis 参数。记法很简单, axis=0 是沿着行往下

```

1 b = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(b.sum()) # 21 不指定就是全部加起来
4 print(b.sum(axis=0)) # [5 7 9] 每列的和
5 print(b.sum(axis=1)) # [6 15] 每行的和

```

开头算每人总分用 axis=1, 因为一个人是一行; 算每科平均用 axis=0, 因为一门课是一列。这个参数用错是最常见的问题, 算之前想清楚结果应该有几个数。

9.4 取值和筛选

本节任务: 会从数组里取出想要的部分, 尤其是按条件筛选。

一维取值和列表一样。二维要给两个下标, 先行后列:

```

1 b = np.array([[1, 2, 3],
2               [4, 5, 6]])
3 print(b[0, 2]) # 3 第 0 行第 2 列
4 print(b[1]) # [4 5 6] 整个第 1 行
5 print(b[:, 0]) # [1 4] 所有行的第 0 列
6 print(b[:, 1:]) # 所有行, 第 1 列往后

```

冒号表示这一维全要, 这个写法在处理表格数据时天天用。

更有用的是按条件筛选。数组和一个数比较, 会逐个元素比较, 得到一个由 True ★核心和 False 组成的数组, 拿它当下标就能选出满足条件的元素:

```
1 a = np.array([86, 92, 55, 78, 90])
2 print(a >= 60) # [ True True False True True]
3 print(a[a >= 60]) # [86 92 78 90] 只留及格的
4 print((a >= 60).sum()) # 4 数一数几个及格
5 print(a[a >= 60].mean())# 86.5 及格者的平均分
```

用 True 和 False 求和能直接数出满足条件的个数，因为 True 当 1 用。开头统计及格人次就是这么算的。

条件可以组合，注意要用 & 和 \，而且每个条件要用括号括起来：

```
1 print(a[(a >= 60) & (a < 90)]) # [86 78] 及格但没到 90 的
```

这里不能用 and 和 or，那两个是给单个真假值用的，数组要用 & 和 竖线。

9.5 广播，形状不同也能算

本节任务：理解形状不一样的数组为什么也能相加。

前面 `np.array([1,2,3]) + 10` 其实就是**广播**。10 是一个数，却和三个元素都相加了，相当于被自动扩展成了 `[10,10,10]`。

二维和一维之间也能广播。比如给每门课加不同的分数：

```
1 scores = np.array([[86, 92, 78],
2                     [95, 88, 91]])
3 bonus = np.array([2, 0, 5]) # 语文加 2，数学不加，英语加 5
4 print(scores + bonus)
```

输出：

```
[[88 92 83]
 [97 88 96]]
```

bonus 只有三个数，却给两行都加上了，它沿着行方向被复制了一遍。广播的规则是从末尾维度开始比对，长度相同或其中一个是 1 就能对上。用不着背规则，实际用的时候形状对不上会报错，报错信息里会写清楚两个形状是什么，照着调整即可。

9.6 常用函数速查

本节任务：认识最常用的一批函数，需要时想得起来。

表 9.1: 常用的数组函数。

写法	干什么
<code>np.zeros(n)</code> 和 <code>np.ones(n)</code>	造一串 0 或 1
<code>np.arange(a, b, step)</code>	像 <code>range</code> 那样造等差数列
<code>np.linspace(a, b, n)</code>	在 <code>a</code> 到 <code>b</code> 之间等分取 <code>n</code> 个数
<code>np.random.rand(n)</code>	<code>n</code> 个 0 到 1 之间的随机数
<code>a.sum()</code> <code>a.mean()</code> <code>a.std()</code>	求和、平均、标准差
<code>a.min()</code> <code>a.max()</code>	最小值、最大值
<code>a.argmax()</code> <code>a.argmin()</code>	最小值、最大值在第几个位置
<code>np.sort(a)</code>	排序
<code>a.T</code>	转置，行列对调

`argmax` 特别有用，它给的是位置而不是值，配合别的数组就能查出对应的信息。开头找总分最高的人就是先 `argmax` 拿到位置，再用这个位置去 `names` 里取名字。

9.7 小结

这一章把成批的数字交给了数组。数组能整体运算，一句话完成过去要循环的事；元素类型必须一致，换来了速度；数组有形状，可以是多维的。`reshape` 改形状，某一维写 -1 让它自己算。求和求平均要指定 `axis`，`axis=0` 得到每列的结果，`axis=1` 得到每行的结果。二维取值先行后列，冒号表示整维全要。用条件筛选能直接选出满足要求的元素，组合条件用 `&` 和竖线并各自加括号。形状不同的数组靠广播也能运算。下一章用表格工具处理带列名的真实数据。

本章知识点

- **数组**——NumPy 的核心结构，能整体运算，元素类型必须一致
- **整体运算**——数组乘 2 是每个元素乘 2，列表乘 2 是重复自己
- **形状**——数组的行列结构，用 `shape` 查看，`reshape` 修改
- **`reshape` 的 -1**——让某一维自动算出来，元素总数必须对得上
- **`axis`**——0 沿行往下走得到每列结果，1 沿列往右走得到每行结果
- **二维取值**——先行后列，冒号表示这一维全要
- **条件筛选**——数组和数比较得到真假数组，拿它当下标选出元素
- **真假求和**——True 当 1 用，求和就是数满足条件的个数
- **组合条件**——用 `&` 和竖线，每个条件要加括号，不能用 `and` `or`
- **广播**——形状不同的数组按规则自动扩展后运算
- **`argmax`**——返回最大值所在的位置而不是值本身